

Plan de Taller de 4 Semanas

Construcción paso a paso de una plataforma de sensado y monitoreo espectral

Propuesta de formación técnica

2026-05-21

Resumen ejecutivo

Este documento propone un taller de 4 semanas para que un equipo técnico comprenda y pueda construir, paso a paso, una plataforma de sensado y monitoreo espectral. El taller no está planteado como una demostración de una plataforma ya terminada, sino como una experiencia guiada para entender su arquitectura, sus módulos, sus contratos de datos y su estilo de implementación.

El enfoque será *AI-driven*: se usará inteligencia artificial como apoyo para levantar requisitos, crear código base, documentar, revisar consistencia, preparar pruebas, depurar errores y explicar decisiones técnicas. No se contempla todavía IA embebida ni modelos corriendo en el sensor; la IA se usa como herramienta de trabajo durante el diseño y la construcción de la plataforma.

El stack se menciona solo como referencia práctica para construir el taller, no como el centro de la formación:

- **Frontend:** Vite + React.
- **Backend:** Node.js con Express, pensado para operación multiusuario.
- **Tiempo real:** WebSocket o una capa equivalente para actualización en vivo.
- **Persistencia:** base de datos relacional, preferiblemente PostgreSQL.
- **Hardware:** cualquier SDR y cualquier embebido disponible para adquisición o simulación de datos.
- **Postprocesamiento:** Python, tratado como módulo externo y con baja profundidad, porque requiere más conocimientos de DSP y no hace parte del núcleo de la plataforma.

Propósito del taller

El propósito es que los participantes puedan entender cómo se diseña y se construye una plataforma que conecta una interfaz web, un backend, una base de datos, sensores o simuladores, hardware SDR/embebido y un módulo externo de análisis.

Al terminar, el equipo debe poder:

- Explicar la arquitectura general de la plataforma.
- Identificar responsabilidades de frontend, backend, base de datos, sensor, embebido y postprocesamiento.
- Entender bases mínimas de RF y SDR para que el sensor no sea una caja negra.

- Levantar un entorno base usando herramientas web modernas.
- Entender el flujo de datos desde el SDR o simulador hasta la interfaz web.
- Diseñar endpoints, eventos en tiempo real y modelos de datos básicos.
- Aplicar un estilo de código ordenado y mantenible.
- Usar IA de forma responsable para acelerar requisitos, código, pruebas y documentación.
- Presentar una demo funcional o semi-funcional de punta a punta.

Audiencia objetivo

El taller está dirigido a un equipo técnico pequeño:

- Un ingeniero de sistemas.
- Dos o más tecnólogos o perfiles técnicos equivalentes.

No se asume dominio previo de radiofrecuencia, DSP o procesamiento avanzado de señales. Sí se recomienda que los participantes tengan bases de programación, uso de terminal, Git, HTTP y desarrollo web básico.

Enfoque pedagógico

El taller se centra en aprender a construir y razonar la plataforma, no solo en verla funcionando. Cada semana combina explicación arquitectónica, revisión de estilo de código, laboratorios guiados y entregables verificables.

La profundidad del código será selectiva:

- Se revisarán estructuras, patrones y responsabilidades.
- Se construirán esqueletos funcionales.
- Se explicarán contratos de API, eventos, modelos y flujos.
- No se hará una explicación exhaustiva de cada línea.
- Se darán bases pequeñas de RF/SDR para entender antenas, frecuencia, potencia, espectro y representación de datos.
- Se evitará profundizar en DSP avanzado.
- Se usará IA para generar borradores, revisar consistencia y documentar decisiones.

Rol de la inteligencia artificial

El enfoque *AI-driven* busca que los participantes aprendan a usar IA como copiloto técnico durante la creación de requisitos, código, documentación, pruebas y material de soporte. La IA no reemplaza el criterio del equipo; ayuda a acelerar ciclos de diseño, documentación y validación.

Este taller no propone todavía IA dentro del embebido, del SDR o del sensor. El hardware se mantiene como fuente de adquisición de datos o como punto de envío hacia la plataforma.

Usos esperados de IA

- Convertir requisitos en historias técnicas.
- Proponer diagramas de arquitectura.
- Generar esqueletos de frontend y backend.
- Proponer modelos de datos iniciales.
- Revisar consistencia entre endpoints, DTOs y componentes.
- Sugerir pruebas manuales y automáticas.
- Explicar errores de compilación o ejecución.
- Documentar decisiones técnicas.
- Preparar guías de operación y handover.
- Generar ejemplos de prompts para mejorar código, requisitos y documentación.

Reglas de uso de IA durante el taller

- Todo código generado por IA debe ser revisado por una persona.
- La IA debe recibir contexto claro: objetivo, archivo, restricción y salida esperada.
- No se aceptan respuestas generadas sin validar contra el flujo real de la plataforma.
- Se debe pedir a la IA que explique supuestos y riesgos.
- Se debe conservar un registro breve de decisiones importantes.
- No se diseña ningún componente de IA embebida durante este taller.

Requisitos de hardware

El taller debe ser flexible frente al hardware disponible. No se amarra a una referencia específica de SDR o embebido.

- Cualquier SDR disponible, por ejemplo RTL-SDR, HackRF, USRP, LimeSDR, SDRplay u otro compatible.
- Cualquier embebido o equipo de borde disponible, por ejemplo Raspberry Pi, Jetson, Beagle-Bone, mini-PC industrial u otro equipo capaz de capturar, preparar o enviar datos.
- Antena compatible con el rango de prueba.
- Computadores para los participantes.
- Red local o acceso a internet para conectar frontend, backend y equipo de adquisición.
- En caso de no tener hardware listo, se usará simulador de datos.

Requisitos de software

- Node.js LTS.
- npm, pnpm o yarn.

- Vite + React para el frontend.
- Node.js + Express para el backend.
- TypeScript recomendado para frontend y backend.
- PostgreSQL local o en contenedor.
- Docker opcional para simplificar servicios.
- Git.
- Cliente HTTP: Postman, Insomnia, Thunder Client o equivalente.
- Python 3.10+ para postprocesamiento o simuladores auxiliares.
- Drivers o librerías necesarias para el SDR elegido.
- Acceso a una herramienta de IA para asistencia técnica.

Arquitectura de referencia

La plataforma se entiende como un sistema por capas:

1. **Capa de adquisición:** SDR y embebido capturan o simulan datos de campo.
2. **Módulo de adquisición:** script o proceso simple que prepara y envía estado, ubicación, espectro y audio cuando aplique.
3. **Backend Express:** coordina usuarios, sensores, configuraciones, campañas, datos, alertas y reportes.
4. **Base de datos:** guarda configuración, histórico, campañas, mediciones y resultados.
5. **Tiempo real:** WebSocket transmite estados y datos recientes al frontend.
6. **Frontend React:** muestra mapas, sensores, monitoreo, campañas, alertas y resultados.
7. **Postprocesamiento Python:** módulo externo para análisis especializado, tratado como integración puntual.

Estilo de código esperado

El taller debe enseñar un estilo de código ordenado, legible y extensible. La meta es que el equipo pueda continuar la plataforma después del taller.

Backend

- Separar rutas, controladores, servicios, repositorios y modelos.
- Mantener controladores delgados: reciben, validan y delegan.
- Concentrar reglas de negocio en servicios.
- Usar DTOs o schemas para entradas y salidas.
- Centralizar configuración de entorno.
- Manejar errores de forma consistente.
- Diseñar endpoints con contratos claros antes de implementar.
- Pensar desde el inicio en multiusuario: sesiones, roles, permisos y auditoría.

Frontend

- Separar páginas, componentes, hooks, servicios API y estado global.
- Mantener componentes de UI simples y reutilizables.
- Evitar lógica de negocio pesada dentro de componentes visuales.
- Definir estados de carga, error, vacío y éxito.
- Documentar flujos de usuario antes de construir pantallas.
- Priorizar una interfaz clara para operación técnica.

Integraciones

- Definir contratos de datos antes de conectar hardware.
- Simular datos antes de depender del SDR real.
- Versionar cambios en endpoints y eventos.
- Registrar logs útiles para diagnóstico.
- Documentar supuestos de frecuencia, resolución, timestamp y unidades.

Duración sugerida

Se propone una duración de 4 semanas con 3 sesiones por semana, cada una de 2 horas. Esto da 24 horas sincrónicas. Se recomienda complementar con 2 a 4 horas semanales de práctica autónoma.

- **Semanas:** 4.
- **Sesiones por semana:** 3.
- **Duración por sesión:** 2 horas.
- **Total sincrónico:** 24 horas.
- **Modalidad:** presencial, remota o mixta.
- **Formato:** explicación breve, laboratorio guiado, discusión de arquitectura y cierre con entregable.

Cronograma detallado

Semana	Sesión	Contenido y actividad	Entregable
1	Sesión 1: visión, RF y SDR básico	Presentación del problema y bases mínimas de radio: espectro electromagnético, RF, antenas, potencia, frecuencia, ancho de banda y qué entrega un SDR. El objetivo es que el sensor no se trate como una caja negra.	Glosario RF/SDR básico y primer mapa del flujo de datos.

Semana	Sesión	Contenido y actividad	Entregable
1	Sesión 2: arquitectura y estilo de proyecto	Actores del sistema, flujo desde sensor hasta interfaz, módulos principales y responsabilidades. Mención breve del stack de referencia. Uso de IA para convertir el problema en arquitectura, requisitos iniciales y lista de capacidades.	Mapa inicial de arquitectura, estructura base propuesta y guía corta de estilo.
1	Sesión 3: entorno y primer flujo mínimo	Preparación del ambiente. Creación del frontend base, backend base, endpoint de salud y cliente de prueba. Uso de IA para generar checklist de instalación, diagnóstico y documentación inicial.	Frontend y backend ejecutándose localmente con primer endpoint probado.
2	Sesión 4: modelo de datos	Diseño de entidades: usuario, sensor, antena, configuración, medición, campaña, alerta y reporte. Relación entre datos operativos e históricos. Discusión de PostgreSQL y migraciones.	Modelo entidad-relación inicial y lista de tablas mínimas.
2	Sesión 5: API Express multiusuario	Diseño de API REST. Separación de rutas, controladores, servicios y repositorios. Sesiones, roles y permisos a nivel conceptual. Construcción de endpoints base para sensores y campañas.	Contrato API inicial y backend modular.
2	Sesión 6: recepción de datos y tiempo real	Diseño del contrato del sensor: estado, GPS, espectro y audio opcional. Ingesta por HTTP y publicación por WebSocket. Simulación de datos antes de usar hardware real.	Flujo backend capaz de recibir datos simulados y emitir eventos en vivo.
3	Sesión 7: arquitectura frontend	Estructura del frontend con la herramienta de referencia: rutas, layout, menú, páginas, componentes, servicios API y manejo de estado. Definición de pantallas principales: inicio, dispositivos, monitoreo, campañas, alertas y configuración.	Mapa de navegación y esqueleto de pantallas.
3	Sesión 8: visualización e interacción	Conexión con API. Estados de carga, error y datos vacíos. Visualización de sensores, estado general, mediciones y campañas. Uso de IA para revisar consistencia entre flujo de usuario y componentes.	Frontend conectado al backend con datos de prueba.

Semana	Sesión	Contenido y actividad	Entregable
3	Sesión 9: monitoreo en vivo	Integración de WebSocket. Actualización de datos en vivo. Vista de monitoreo, selección de sensor, parámetros de medición y lectura de datos recientes. Discusión de estilo visual operativo.	Pantalla de monitoreo funcional con datos simulados o reales.
4	Sesión 10: SDR, embebido y adquisición	Conexión conceptual con cualquier SDR y cualquier embebido. Diseño del flujo de adquisición: captura o simulación, empaquetado de datos, envío al backend y manejo de errores. Si no hay hardware, se usa simulador.	Contrato de adquisición y prueba de envío desde hardware o simulador.
4	Sesión 11: reportes y postprocesamiento Python	Explicación liviana del rol de Python: análisis externo, generación de resultados y contrato con backend. Se recalca que DSP no es el centro del taller. Diseño de flujo para reporte o resultado de campaña.	Contrato de integración con Python y flujo de reporte definido.
4	Sesión 12: integración final y handover	Prueba punta a punta: sensor o simulador, backend, base de datos, WebSocket, frontend y reporte básico. Revisión de riesgos, documentación mínima, tareas pendientes y demo final.	Demo final, guía de ejecución y backlog de continuidad.

Plan por semanas

Semana 1: fundamentos RF/SDR, arquitectura y entorno

Objetivo: que el equipo entienda qué se va a construir, cómo fluye una medición desde el mundo RF hasta la interfaz y por qué la plataforma se separa en módulos.

Temas principales:

- Propósito de la plataforma.
- Bases del espectro electromagnético y radiofrecuencia.
- Antenas: rol, bandas, ganancia, polarización y relación con la medición.
- SDR: qué mide, qué no mide y cómo entrega datos.
- Representación de datos: frecuencia, potencia, muestras, FFT, resolución y ancho de banda a nivel introductorio.
- Diferencia entre señal física, dato capturado y dato visualizado.
- Flujo desde SDR/embebido hasta frontend.
- Diferencia entre demostración, prototipo y plataforma multiusuario.
- Arquitectura por capas.

- Stack tecnológico como referencia, sin convertirlo en el foco del curso.
- Reglas de estilo de código.
- Uso de IA para requisitos, arquitectura, documentación y generación de esqueletos.

Laboratorios:

- Construir un glosario mínimo de RF/SDR.
- Dibujar el recorrido de una señal desde antena hasta pantalla.
- Crear diagrama de arquitectura asistido por IA.
- Crear estructura base del proyecto.
- Levantar el frontend con la herramienta de referencia.
- Levantar el backend con la herramienta de referencia.
- Crear endpoint de salud.
- Probar backend desde cliente HTTP.

Producto verificable: repositorio base ejecutándose localmente, glosario RF/SDR, diagrama inicial y guía de estilo.

Semana 2: backend, datos y tiempo real

Objetivo: construir la columna vertebral de la plataforma: API, modelos, persistencia, recepción de datos y eventos en vivo.

Temas principales:

- Diseño de datos.
- Backend modular con Express.
- Multiusuario: usuarios, roles, permisos y sesiones.
- Gestión de sensores y antenas.
- Gestión de campañas.
- Recepción de datos de sensor.
- Eventos en tiempo real.
- Pruebas básicas de endpoints.
- Uso de IA para generar código base, revisar contratos y documentar endpoints.

Laboratorios:

- Diseñar tablas principales.
- Crear endpoints base.
- Crear un simulador simple de sensor.
- Recibir datos de estado, ubicación y espectro.
- Publicar eventos por WebSocket.
- Documentar contratos de API.

Producto verificable: backend funcional con base de datos, endpoints principales y flujo de datos simulado.

Semana 3: frontend e integración

Objetivo: construir la experiencia web que permita operar, consultar y visualizar la plataforma.

Temas principales:

- Arquitectura de frontend con la herramienta definida para el taller.
- Rutas, layout y menú.
- Servicios API.
- Manejo de estado.
- Pantallas de inicio, dispositivos, monitoreo, campañas, alertas y configuración.
- Visualización de datos en vivo.
- Estados de interfaz: cargando, error, vacío, éxito.
- Uso de IA para crear componentes base, revisar estados de interfaz y documentar decisiones.

Laboratorios:

- Crear layout principal.
- Crear servicios de consumo de API.
- Mostrar sensores y campañas.
- Conectar vista de monitoreo a WebSocket.
- Revisar consistencia visual y técnica con IA.

Producto verificable: frontend conectado al backend, con flujo de monitoreo visible y datos simulados o reales.

Semana 4: hardware, postprocesamiento liviano, pruebas y entrega

Objetivo: cerrar un flujo completo, integrar hardware o simulador y preparar una demo final entendible.

Temas principales:

- Conexión conceptual con cualquier SDR.
- Uso de cualquier embebido como nodo de adquisición.
- Módulo simple de captura o envío de datos.
- Contrato entre adquisición y backend.
- Postprocesamiento Python como módulo externo.
- Reportes y resultados.
- Pruebas end-to-end.
- Documentación de operación.

- Handover técnico.

Laboratorios:

- Enviar datos desde SDR/embebido o simulador.
- Validar recepción en backend.
- Ver actualización en frontend.
- Ejecutar flujo de campaña o monitoreo.
- Generar resultado o reporte básico.
- Preparar demo final.

Producto verificable: demo de punta a punta, documentación mínima, lista de riesgos y backlog de continuidad.

Alcance del postprocesamiento Python

El postprocesamiento se menciona y se integra de forma controlada, pero no será el centro del taller. La razón es que el análisis de señales y DSP requiere conocimientos especializados adicionales.

Durante el taller se verá:

- Qué recibe Python desde el backend.
- Qué devuelve Python al backend.
- Cómo se invoca el módulo externo.
- Cómo se presenta un resultado en la interfaz.
- Qué límites tiene el análisis si no se entra en DSP.

No se profundizará en:

- Teoría avanzada de FFT.
- Detección robusta de emisiones.
- Demodulación avanzada.
- Calibración de RF.
- Modelos matemáticos de cumplimiento.

Estructura sugerida del repositorio

La estructura exacta puede cambiar, pero el taller debe dejar claro cómo separar responsabilidades.

```
platform/  
  frontend/  
    src/  
      components/  
      pages/  
      hooks/
```

```
    services/  
    state/  
    styles/  
backend/  
  src/  
    routes/  
    controllers/  
    services/  
    repositories/  
    models/  
    websocket/  
    config/  
acquisition-client/  
  src/  
    capture/  
    transport/  
    config/  
postprocessing/  
  src/  
    adapters/  
    reports/  
docs/  
  architecture/  
  api/  
  runbooks/
```

Entregables finales

Al finalizar las 4 semanas se espera entregar:

- Diagrama de arquitectura.
- Guía de instalación y ejecución.
- Repositorio base o estructura de referencia.
- Backend Express con endpoints principales.
- Frontend con pantallas principales usando la herramienta definida para el taller.
- Flujo de datos desde simulador, SDR o embebido.
- WebSocket o mecanismo equivalente de datos en vivo.
- Contrato básico para postprocesamiento Python.
- Demo final.
- Backlog de mejoras.

Criterios de éxito

El taller será exitoso si los participantes pueden:

- Explicar la plataforma sin depender del instructor.

- Identificar dónde vive cada responsabilidad técnica.
- Crear un nuevo endpoint siguiendo el estilo del backend.
- Crear una nueva pantalla siguiendo el estilo del frontend.
- Conectar una fuente de datos simulada o real.
- Diagnosticar errores básicos de API, base de datos y WebSocket.
- Usar IA para acelerar tareas sin copiar ciegamente.
- Presentar una demo técnica coherente.

Riesgos y mitigaciones

- **Hardware no disponible o inestable:** usar simulador como plan base y dejar SDR/embebido como integración progresiva de adquisición.
- **Exceso de profundidad en código:** mantener el foco en arquitectura, contratos y estilo.
- **Dificultad de DSP:** tratar Python como caja externa con contrato claro.
- **Dependencia excesiva de IA:** exigir revisión humana y explicación de decisiones.
- **Diferencias de nivel técnico:** trabajar por parejas y dejar tareas de refuerzo.
- **Alcance demasiado grande:** priorizar flujo mínimo de punta a punta antes de funcionalidades avanzadas.

Agenda sugerida de cada sesión

Cada sesión de 2 horas puede seguir este formato:

- 15 minutos: objetivo y repaso del flujo.
- 25 minutos: explicación arquitectónica.
- 45 minutos: laboratorio guiado.
- 20 minutos: revisión con IA y discusión de estilo.
- 15 minutos: cierre, dudas y tarea corta.

Material mínimo para el instructor

- Diagrama de arquitectura base.
- Repositorio inicial o plantilla.
- Prompts guía para IA.
- Colección de endpoints de prueba.
- Simulador de sensor.
- Datos de ejemplo.
- Checklist de instalación.
- Checklist de demo final.

Conclusión

Este taller de 4 semanas busca que el equipo aprenda a construir la plataforma paso a paso, entendiendo primero bases mínimas de RF/SDR y la arquitectura, y luego bajando a implementaciones guiadas. El stack de referencia permite cubrir el flujo completo sin convertir el taller en una clase extensa de DSP ni en una revisión exhaustiva de código.

El resultado esperado es un equipo capaz de explicar, ejecutar, modificar y continuar la plataforma con criterio técnico, apoyo de IA para producir código/documentación/requisitos, y una base de trabajo ordenada.